2023 IEEE/CVF International Conference on Computer Vision (ICCV) | 979-8-3503-0718-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICCV51070.2023.00499

Poincaré ResNet

Max van Spengler[1], Erwin Berkhout[2], Pascal Mettes[1]

[1] VIS Lab, Informatics Institute, University of Amsterdam,
[2] Department of Oral Radiology, Academic Center for Dentistry, University of Amsterdam & VU Amsterdam

Abstract

This paper introduces an end-to-end residual network that operates entirely on the Poincaré ball model of hyperbolic space. Hyperbolic learning has recently shown great potential for visual understanding, but is currently only performed in the penultimate layer(s) of deep networks. All visual representations are still learned through standard Euclidean networks. In this paper we investigate how to learn hyperbolic representations of visual data directly from the pixel-level. We propose Poincaré ResNet, a hyperbolic counterpart of the celebrated residual network, starting from Poincaré 2D convolutions up to Poincaré residual connections. We identify three roadblocks for training convolutional networks entirely in hyperbolic space and propose a solution for each: (i) Current hyperbolic network initializations collapse to the origin, limiting their applicability in deeper networks. We provide an identity-based initialization that preserves norms over many layers. (ii) Residual networks rely heavily on batch normalization, which comes with expensive Fréchet mean calculations in hyperbolic space. We introduce Poincaré midpoint batch normalization as a faster and equally effective alternative. (iii) Due to the many intermediate operations in Poincaré layers, the computation graphs of deep learning libraries blow up, limiting our ability to train on deep hyperbolic networks. We provide manual backward derivations of core hyperbolic operations to maintain manageable computation graphs.

1. Introduction

Deep learning in hyperbolic space has gained traction in recent years empowered by their inherent ability to embed hierarchical data with arbitrarily low distortion [45] and by being more compact and dense [5, 42, 47]. These promising characteristics have led to rapid developments in hyperbolic representation learning for tree-like structures [1, 4, 15, 27, 42, 44], graphs [5, 9, 29, 60], text [7, 11, 49], action skeletons [14], biological structures [26], and more.

Recently, hyperbolic learning has also been investigated for visual understanding. Hyperbolic embeddings of images and videos have been shown to improve few-shot learning [13, 17, 20, 35, 57], hierarchical recognition [10, 18, 32, 33, 56], segmentation [6, 19] and metric learning [12, 59] amongst others. While promising, the use of hyperbolic geometry in computer vision has been limited to the classifier space, with visual representations being learned on conventional networks that operate in Euclidean space.

This paper explores learning visual representations entirely in hyperbolic space. The ability to learn hyperbolic representations directly from the pixel-level will allow us to unlock the broad potential of hyperbolic geometry for vision, such as capturing latent hierarchical visual representations [25], training compact network architectures [5, 42, 47], and creating networks that better mimic visual representation learning in the brain [58]. Empowered by successful implementations of non-visual layers [16, 47], the time is ripe for visual hyperbolic feature learning.

As a step towards fully hyperbolic visual learning, we start from the highly celebrated ResNet [22] and rebuild its architecture in hyperbolic space; from 2D convolutions to residual connections. Optimizing a ResNet in the Poincaré ball model comes with several challenges. First, we find that existing network initializations in hyperbolic space lead to vanishing signals, which derail learning over many convolutional layers. We provide an identity-based network initialization that preserves the output norm over many layers. Second, ResNets rely extensively on batch normalization, but its generalization to hyperbolic space requires expensive Fréchet mean calculations [34]. We introduce Poincaré midpoint batch normalization, which allows us to compute approximate means at a fraction of the computational cost. Third, the basic gyrovector operations in the Poincaré ball model consist of many intermediate calculations. In modern deep learning libraries, all these calculations are stored for automatic differentiation, blowing up the computation graph. We have derived and implemented the backward pass of core hyperbolic gyrovector operations

2380-7504/23/$31.00 ©2023 IEEE
DOI 10.1109/ICCV51070.2023.00499

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.

to contain the computation graph.

Empirically, we show that our network initialization is indeed norm-preserving and improves network generalization. We show that our midpoint batch normalization speeds up training by 25% with no loss in classification accuracy. We furthermore demonstrate the potential of Poincaré ResNet for out-of-distribution detection, adversarial robustness, and learning complementary representations compared to Euclidean ResNet. The code is available at https://github.com/maxvanspengler/poincare-resnet with a similar implementation in the documentation of HypLL [52].

2. Background and related work

2.1. Poincaré ball model of hyperbolic space

This paper operates on the most commonly used model of hyperbolic geometry in deep learning, namely the Poincaré ball model. We will therefore restrict the background discussion to this model and refer to Peng *et al*. [43] for a more comprehensive discussion on the different isometric models of hyperbolic space. The n-dimensional Poincaré ball model with constant negative curvature $-c$ is defined as the Riemannian manifold $(\mathbb{B}_c^n, \mathfrak{g}_c)$, where

$$\mathbb{B}_c^n = \{\boldsymbol{x} \in \mathbb{R}^n : ||\boldsymbol{x}||^2 < \frac{1}{c}\}, \tag{1}$$

and where

$$\mathfrak{g}_c = \lambda_{\boldsymbol{x}}^c I_n, \quad \lambda_{\boldsymbol{x}}^c = \frac{2}{1 - c||\boldsymbol{x}||^2}, \tag{2}$$

with I_n being the n-dimensional identity matrix. The Poincaré ball model can be turned into a gyrovector space [50] by endowing it with Möbius addition and Möbius scalar multiplication, respectively defined as

$$\begin{aligned} \boldsymbol{x} \oplus_c \boldsymbol{y} &= \frac{(1 + 2c\langle \boldsymbol{x}, \boldsymbol{y}\rangle + c||\boldsymbol{y}||^2)\boldsymbol{x} + (1 - c||\boldsymbol{x}||^2)\boldsymbol{y}}{1 + 2c\langle \boldsymbol{x}, \boldsymbol{y}\rangle + c^2||\boldsymbol{x}||^2||\boldsymbol{y}||^2}, \\ r \otimes_c \boldsymbol{x} &= \frac{1}{\sqrt{c}} \tanh\left(r \tanh^{-1}(\sqrt{c}||\boldsymbol{x}||)\right)\frac{\boldsymbol{x}}{||\boldsymbol{x}||}, \end{aligned} \tag{3}$$

where $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{B}_c^n$, $r \in \mathbb{R}$ and where $||\cdot||$ and $\langle \cdot, \cdot \rangle$ denote the Euclidean norm and inner product, respectively. An important map related to this gyrovector space is the gyrator $\mathrm{gyr} : \mathbb{B}_c^n \times \mathbb{B}_c^n \to \mathrm{Aut}(\mathbb{B}_c^n, \oplus_c)$, where $\mathrm{Aut}(\mathbb{B}_c^n, \oplus_c)$ denotes the set of automorphisms on $\mathbb{B}_c^n$ [50]. This map is implicitly defined as

$$\mathrm{gyr}[\boldsymbol{x}, \boldsymbol{y}]\boldsymbol{z} = -(\boldsymbol{x} \oplus_c \boldsymbol{y}) \oplus_c \left(\boldsymbol{x} \oplus_c (\boldsymbol{y} \oplus_c \boldsymbol{z})\right), \tag{4}$$

where $\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z} \in \mathbb{B}_c^n$, which can be used to measure the extent to which Möbius addition deviates from commutativity. It will be used later on to define parallel transport. Furthermore, we can compute the distance between any two points $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{B}_c^n$ as

$$d_c(\boldsymbol{x}, \boldsymbol{y}) = \frac{2}{\sqrt{c}} \tanh^{-1}(\sqrt{c}||-\boldsymbol{x} \oplus_c \boldsymbol{y}||). \tag{5}$$

For an in-depth analysis of this gyrovector space approach to the Poincaré ball see [50]. Using the definition of Möbius addition, the exponential and logarithmic maps can be written as [16]

$$\begin{aligned} \exp_{\boldsymbol{x}}^c(\boldsymbol{v}) &= \boldsymbol{x} \oplus_c \left(\tanh\left(\frac{\sqrt{c}\lambda_{\boldsymbol{x}}^c||\boldsymbol{v}||}{2}\right)\frac{\boldsymbol{v}}{\sqrt{c}||\boldsymbol{v}||}\right), \\ \log_{\boldsymbol{x}}^c(\boldsymbol{y}) &= \frac{2}{\sqrt{c}\lambda_{\boldsymbol{x}}^c} \tanh^{-1}\left(\sqrt{c}||-\boldsymbol{x} \oplus_c \boldsymbol{y}||\right)\frac{-\boldsymbol{x} \oplus_c \boldsymbol{y}}{||-\boldsymbol{x} \oplus_c \boldsymbol{y}||}, \end{aligned}$$

where $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{B}_c^n$ and $\boldsymbol{v} \in \mathcal{T}_{\boldsymbol{x}}\mathbb{B}_c^n$. Moreover, we can define parallel transport $P_{\boldsymbol{x} \to \boldsymbol{y}}^c : \mathcal{T}_{\boldsymbol{x}}\mathbb{B}_c^n \to \mathcal{T}_{\boldsymbol{y}}\mathbb{B}_c^n$ as follows [47]

$$P_{\boldsymbol{x} \to \boldsymbol{y}}^c(\boldsymbol{v}) = \frac{\lambda_{\boldsymbol{x}}^c}{\lambda_{\boldsymbol{y}}^c}\mathrm{gyr}[\boldsymbol{y}, -\boldsymbol{x}]\boldsymbol{v}, \tag{6}$$

which allows us to transport a tangent vector at a point $\boldsymbol{x} \in \mathbb{B}_c^n$ to the tangent space at another point $\boldsymbol{y} \in \mathbb{B}_c^n$, used for example in batch normalization.

2.2. The Poincaré ball model in neural networks

To perform deep learning on the Poincaré ball model, Ganea *et al*. [16] outline a theoretical framework for incorporating this model into core layers of neural networks, such as hyperbolic logistic regression, hyperbolic fully-connected, and hyperbolic recurrent layers. More recently, Shimizu *et al*. [47] made important improvements to this framework to ensure that the hyperbolic geometry was fully taken advantage of without the need for additional learnable parameters. We will therefore use this work as a starting point for the rest of this paper and provide a short overview here.

As a foundation, Poincaré multinomial logistic regression is defined by computing the score for each of n classes for some input $\boldsymbol{x} \in \mathbb{B}_c^m$ as

$$\begin{aligned} v_k(\boldsymbol{x}) = \frac{2}{\sqrt{c}}||\boldsymbol{z}_k|| \sinh^{-1}\Big(&\lambda_{\boldsymbol{x}}^c\langle\sqrt{c}\boldsymbol{x}, \frac{\boldsymbol{z}_k}{||\boldsymbol{z}_k||}\rangle \cosh(2\sqrt{c}r_k) \\ &-(\lambda_{\boldsymbol{x}}^c - 1)\sinh(2\sqrt{c}r_k)\Big), \end{aligned}$$

where $\boldsymbol{z}_k \in \mathcal{T}_{\boldsymbol{0}}\mathbb{B}_c^m = \mathbb{R}^m$ and $r_k \in \mathbb{R}$ are the parameters for the k-th class. These scores are equivalent to the distances between the input $\boldsymbol{x}$ and the n different Poincaré hyperplanes determined by the parameters $\{(\boldsymbol{z}_k, r_k)\}_{i=1}^n$. Here, $\boldsymbol{z}_k$ determines the orientation of the hyperplane while r_k determines its offset with respect to the origin. A Poincaré

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.

fully connected layer mapping input $\boldsymbol{x} \in \mathbb{B}_c^m$ to $\mathbb{B}_c^n$ is in turn defined as

$$\boldsymbol{y} = \mathcal{F}^c(\boldsymbol{x}; Z, \boldsymbol{r}) = \frac{\boldsymbol{w}}{1 + \sqrt{1 + c||\boldsymbol{w}||^2}}, \tag{7}$$

with

$$\boldsymbol{w} = \Big(\frac{1}{\sqrt{c}} \sinh(\sqrt{c} v_k(\boldsymbol{x}))\Big)_{k=1}^n, \tag{8}$$

where the $v_k(\cdot)$ are the scores from the Poincaré multinomial logistic regression and where $Z = [\boldsymbol{z}_1 | \dots | \boldsymbol{z}_n] \in (\mathcal{T}_{\mathbf{0}} \mathbb{B}_c^m)^n = \mathbb{R}^{m \times n}$ and $\boldsymbol{r} = (r_k)_{k=1}^n \in \mathbb{R}^m$ are the parameters of the layer. Given hyperbolic fully connected layers, Shimizu *et al*. [47] outline general formulations for self-attention and convolutional operations in hyperbolic space. We take such investigations to the visual domain and arrive at Poincaré ResNets, which require 2D convolutions, fast batch normalization, residual blocks, norm-preserving initialization and derived backpropagation of core operations in order to be realized.

2.3. Hyperbolic learning in computer vision

Khrulkov *et al*. [25] have shown that both image data and labels contain hierarchical structures and introduced Hyperbolic Image Embeddings to exploit these observations. In their approach, embeddings of images from standard networks are mapped to hyperbolic space, followed by a final classification layer based on hyperbolic logistic regression or hyperbolic prototypical learning, directly improving few-shot learning and uncertainty quantification.

A wide range of works have investigated hyperbolic visual embeddings, see Mettes *et al*. [38]. Several works have proposed prototypes-based hyperbolic embeddings for few-shot learning [13, 17, 20, 35, 57], where hyperbolic space consistently outperforms Euclidean space. Hyperbolic embeddings of classes based on their hierarchical relations has also shown to be effective for zero-shot learning [30, 54] and hierarchical recognition [10, 18, 32, 56]. Hyperbolic embeddings have furthermore been effective in metric learning [12, 59], object detection [51], image segmentation [6, 19] and future prediction in videos [48].

In generative learning, hyperbolic variational auto-encoders [23, 36, 40], generative adversarial networks [28] and normalizing flows [3, 37] have been shown to obtain competitive results in data-constrained settings. A number of recent works have proposed unsupervised hyperbolic learning approaches [23, 39, 53, 55], allowing for learning and discovering hierarchical representations.

This body of literature highlights that hyperbolic geometry is fruitful for visual understanding. In current literature, however, hyperbolic learning is restricted to the final embedding layers, with all visual representations being learned by standard networks. This paper strives to learn hyperbolic representations in an end-to-end manner, from pixels to labels, complementing current research on computer vision with hyperbolic embeddings.

3. Poincaré residual networks for images

We consider the problem of image classification where our dataset is denoted by $(\boldsymbol{x}_i, y_i)_{i=1}^N$, with $\boldsymbol{x}_i \in \mathbb{R}^{H \times W \times 3}$ and $y_i \in \{1, \dots, C\}$. Here, $\boldsymbol{x}_i$ denotes the pixel values of the i-th input image with height H and width W, while y_i denotes the corresponding label. Our goal is to train a network $y = \phi(\boldsymbol{x})$ that maps an input image $\boldsymbol{x}$ to a label y. Specifically, we strive to formulate the celebrated ResNet [22] architecture in the Poincaré ball model.

In residual networks, the basic building block consists of two weight layers with a ReLU activation between the layers. Afterwards, the input is added to the transformed output through a residual connection, followed by another ReLU activation. A weight layer is typically given as a convolutional layer followed by a batch normalization. Thus, to create Poincaré residual blocks, all these operations need to be formulated in hyperbolic space. Below, we separately outline how to formalize and construct (i) Poincaré 2D convolutions and residual blocks, (ii) how to initialize hyperbolic networks, (iii) Poincaré midpoint batch normalization, and (iv) forward and backward propagation of core hyperbolic operations.

3.1. Poincaré convolutions and residuals

We start by formalizing 2D convolutional operations for images in the Poincaré ball model using the approach of Shimizu *et al*. [47]. Suppose we have an input image $\boldsymbol{x}$ with pixel values

$$\boldsymbol{x}_{ij} \in \mathbb{B}_c^{C_{in}}, \quad i = 1, \dots, H_{in}, \;\; j = 1, \dots, W_{in}, \tag{9}$$

where C_{in} is the number of input channels and where H_{in} and W_{in} are the height and width of the image, respectively. Then we can define a 2D Poincaré convolution operation with C_{out} output channels and with receptive field size $K \times K$, with K odd. This approach and its Euclidean counterpart have the same grid connections between the input values and output values. Only the convolutional operations behind these connections are defined differently. So, the output will have pixel values

$$\boldsymbol{h}_{ij} \in \mathbb{B}_c^{C_{out}}, \quad i = 1, \dots, H_{out}, \;\; j = 1, \dots, W_{out}, \tag{10}$$

where $\boldsymbol{h}_{kl}$ is computed from the pixels $\boldsymbol{x}_{ij}$ in the receptive field at that position, so where

$$\begin{aligned} k - \Big\lfloor \frac{K}{2} \Big\rfloor \leq i \leq k + \Big\lfloor \frac{K}{2} \Big\rfloor, \\ l - \Big\lfloor \frac{K}{2} \Big\rfloor \leq j \leq l + \Big\lfloor \frac{K}{2} \Big\rfloor. \end{aligned} \tag{11}$$

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.

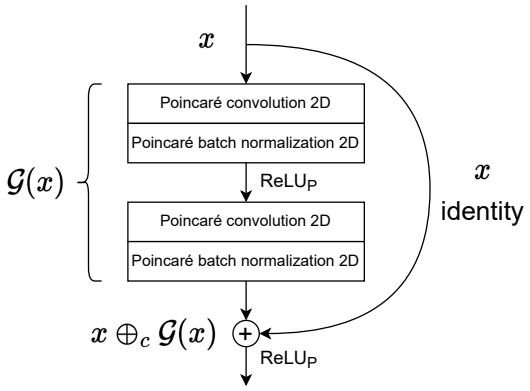


Figure 1: **A Poincaré residual block**, the basic building block of our Poincaré ResNet architectures and a direct generalization of the original residual block of He *et al*. [22].

We denote this receptive field at position (k, l) by X_{kl}. Note that H_{out} and W_{out} depend on the input dimensions, the receptive field size K and, optionally, on stride and padding.

Similar to the Euclidean convolutional layer, for each $\boldsymbol{h}_{kl}$, we want to apply a fully connected layer to the concatenation of the vectors within the receptive field, so we want to compute the output as

$$\boldsymbol{h}_{kl} = \mathcal{F}^c(\|X_{kl}; Z, \boldsymbol{r}), \tag{12}$$

where $\|\cdot$ denotes some concatenation operation and $\mathcal{F}^c$ is the Poincaré fully-connected layer defined in equation (7) with parameters Z and $\boldsymbol{r}$. Note that the usual concatenation is inappropriate for vectors on the Poincaré ball as this can result in vectors outside the manifold. We therefore employ β-concatenation as an alternative, which is a concatenation operation that preserves the expectation of the Poincaré norm of the output vector [47]. This operation, applied to M Poincaré vectors $\{\boldsymbol{b}_i \in \mathbb{B}_c^{n_i}\}_{i=1}^M$ with $n = \sum_i n_i$, is defined in three steps:

1. Map each of the vectors to the tangent space at the origin of their respective Poincaré balls: $\boldsymbol{v}_i = \log_0^c(\boldsymbol{b}_i)$;
2. Let $\beta_n = B(\frac{n}{2}, \frac{1}{2})$, with B the beta function, scale each of the vectors $\boldsymbol{v}_i$ by $\beta_n \beta_{n_i}^{-1}$ and let $\boldsymbol{v}$ be the concatenation of these scaled vectors, so $\boldsymbol{v} = (\beta_n \beta_{n_1}^{-1} \boldsymbol{v}_1^T, \ldots, \beta_n \beta_{n_N}^{-1} \boldsymbol{v}_N^T)^T$;
3. Project the resulting vector back onto the n-dimensional Poincaré ball: $\exp_0^c(\boldsymbol{v})$.

We denote this operation by ${}_\beta\|\cdot$. Now, we can write the 2D Poincaré convolution operation as

$$\boldsymbol{h}_{kl} = \mathcal{F}^c({}_\beta\|X_{kl}; Z, \boldsymbol{r}), \tag{13}$$

where $k = 1, \ldots, H_{out}$, $l = 1, \ldots, W_{out}$ and where $\mathcal{F}^c$ maps from $\mathbb{B}_c^{K^2 \times C_{in}}$ to $\mathbb{B}_c^{C_{out}}$.

Algorithm 1 Poincaré midpoint batch normalization

Training Input: Data batches $\{\boldsymbol{x}_1^{(t)}, \ldots, \boldsymbol{x}_m^{(t)}\} \subseteq \mathbb{B}_c^n$ for $t \in [1, \ldots, T]$, testing momentum $\eta \in [0, 1]$
Learned Parameters: $\beta \in \mathbb{B}_c^n$, $\gamma \in \mathbb{R}$
Normalization Algorithm:
for $t = 1, \ldots, T$ **do**
  $\mu \leftarrow \text{PoincaréMidpoint}(\{\boldsymbol{x}_1^{(t)}, \ldots, \boldsymbol{x}_m^{(t)}\})$
  $\sigma^2 \leftarrow \frac{1}{m} \sum_{i=1}^m d(\boldsymbol{x}_i^{(t)}, \mu)^2$
  for $i = 1, \ldots, m$ **do**
    $\tilde{\boldsymbol{x}}_i^{(t)} \leftarrow \exp_\beta^c \left(\sqrt{\frac{\gamma}{\sigma^2}} P_{\mu \to \beta}^c (\log_\mu^c \boldsymbol{x}_i^{(t)}) \right)$
  end for
  return normalized batch $\tilde{\boldsymbol{x}}_1^{(t)}, \ldots, \tilde{\boldsymbol{x}}_m^{(t)}$
end for

Next, we define a Poincaré version of the residual block by replacing the convolutional layers by Poincaré convolutional layers and by applying a hyperbolic batch normalization, which will be defined in the next subsection. Pointwise nonlinearities can still be applied in the tangent space at the origin of the Poincaré ball by using the logarithmic and exponential maps. So, the Poincaré version of the ReLU nonlinearity becomes

$$\text{ReLU}_P = \exp_0^c \circ \text{ReLU} \circ \log_0^c, \tag{14}$$

where $\circ$ denotes function composition. We will use this Poincaré version to replace the two ReLU nonlinearities. We can furthermore replace the skip connection by $\boldsymbol{x} \oplus_c \mathcal{G}(\boldsymbol{x})$, where $\mathcal{G}$ denotes the transformation given by the two Poincaré convolutional layers and Poincaré batch normalizations. Figure 1 visualizes the Poincaré residual block.

3.2. Poincaré midpoint batch normalization

In a residual block, each convolutional layer is immediately followed by a batch normalization step. Lou *et al*. [34] have previously defined a Poincaré version of batch normalization based on their iterative approximation to the Fréchet mean. While more efficient than previously available methods, this iterative approach still makes the Fréchet mean a computationally expensive step. Directly plugging a Fréchet-based batch normalization in our Poincaré ResNet would account for roughly 77% of the computation in a forward step. We therefore seek to perform batch normalization with greater computational efficiency.

We suggest to take an alternative aggregation of Poincaré vectors, namely the Poincaré midpoint [50]. The midpoint of the Poincaré vectors $\{\boldsymbol{x}_i \in \mathbb{B}_c^n\}_{i=1}^N$ is computed as

$$\mu = \frac{1}{2} \otimes_c \frac{\sum_{i=1}^N \lambda_{\boldsymbol{x}_i}^c \boldsymbol{x}_i}{\sum_{i=1}^N (\lambda_{\boldsymbol{x}_i}^c - 1)}. \tag{15}$$

The resulting midpoint batch normalization algorithm is outlined in Algorithm 1. The goal of batch normalization

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.

is to keep feature vectors centered around the origin and to keep the variance of their norms within a manageable range. By replacing the Fréchet mean by the Poincaré midpoint, the vectors will no longer be centered exactly at the origin, but still close enough to achieve the improved stability that batch normalization normally results in. Moreover, the Poincaré midpoint can be computed directly without any iterative methods, making it substantially faster to compute than the Fréchet mean.

3.3. Hyperbolic network initialization

The canonical ResNet architecture uses Kaiming initialization, which aims to prevent reduction or magnification of input signals as this would hinder convergence during training [21]. This is achieved by maintaining the variance of the components of both the features and the gradients throughout the network. However, such an approach is inappropriate for the Poincaré fully connected and convolutional layers as the components of a Poincaré vector are necessarily dependent, since the Euclidean norm of such vectors is bounded by $c^{-\frac{1}{2}}$.

To that end, Shimizu *et al*. [47] propose to initialize the weights Z of the Poincaré fully connected layer through sampling from the normal distribution $\mathcal{N}(0, (2mn)^{-1})$, where m is the input dimension and n the output dimension of the layer. The biases r are initialized as zeros. We find that this initialization results in vanishing signals, where the norm of an input converges to zero after a few layers. To obtain a norm-preserving network initialization in hyperbolic space, we take the initialization for the weights of a Poincaré layer mapping from $\mathbb{B}_c^m$ to $\mathbb{B}_c^n$ with $m \leq n$ as

$$Z = \begin{cases} \frac{1}{2} I_n & m = n, \\ \frac{1}{2}[I_m | O_{m,n-m}] & m < n, \end{cases} \tag{16}$$

where I_n is the $n \times n$-identity matrix and where $O_{i,j}$ is the $i \times j$-zero matrix. We initialize the biases r as a vector of zeros. Using this initialization, for $m = n$, we see that

$$v_k(\boldsymbol{x}) = \frac{1}{\sqrt{c}} \sinh^{-1}\left(\sqrt{c}\lambda_{\boldsymbol{x}}^c x_k\right), \tag{17}$$

and, therefore,

$$w = \lambda_{\boldsymbol{x}}^c \boldsymbol{x}, \tag{18}$$

from which it follows that $\boldsymbol{y} = \boldsymbol{x}$. When $m < n$, we get $\boldsymbol{y} = (\boldsymbol{x}^T | \mathbf{0}_{n-m}^T)^T$ instead, where $\mathbf{0}_{n-m}$ is an $(n-m)$-dimensional vector of zeros. Thus, for the cases $m \leq n$, this initialization keeps the norms of the vectors constant throughout the network.

For residual networks, $m \leq n$ for each layer except for the linear layer at the end of the network. Therefore, we initialize each of the convolutional layers using our identity initialization. The final linear layer will be initialized using the initialization by [47]. We find the vanishing effect of this single layer to be harmless to the performance.

3.4. Optimization and backward propagation

For neural networks on Riemannian manifolds, one generally has to consider the manifold on which the parameters live for optimization [2]. For Poincaré residual networks, we need to consider the weights of three different layers, namely, the fully-connected layer, the convolutional layer, and the batch normalization. The parameters of the fully-connected layer and the convolutional layer as proposed by Shimizu *et al*. [47] live in Euclidean space, so we can use Euclidean optimizers for these layers. However, the batch normalization algorithm shown in Algorithm 1 makes use of a parameter vector living on $\mathbb{B}_c^n$. To avoid difficulties with optimizers, we instead supply the algorithm with a parameter vector in $\mathbb{R}^n$ that is mapped to the Poincaré ball using the exponential map around the origin. This is used by Lou *et al*. [34] as well. As a result, we can optimize Poincaré residual networks using traditional Euclidean optimizers.

A direct consequence of applying hyperbolic operations in a neural network is the large computational cost incurred by the many applications of nonlinear operations. This leads to a significant increase in memory requirements as all these intermediate steps become part of the computation graph during training. To maintain compact computation graphs, we have manually derived the backward pass of several core hyperbolic operations, namely Möbius addition, the exponential and logarithmic maps and the conformal factor $\lambda_{\boldsymbol{x}}^c$. The use of these manually defined derivatives also reduces the size of the computation graph of many other operations defined on the Poincaré ball, as these generally build upon the more basic operations. We find that using manually defined derivatives reduces memory usage by approximately 30%, but increases computation time. Due to the length of the derivations, we provide a full breakdown in the supplementary materials.

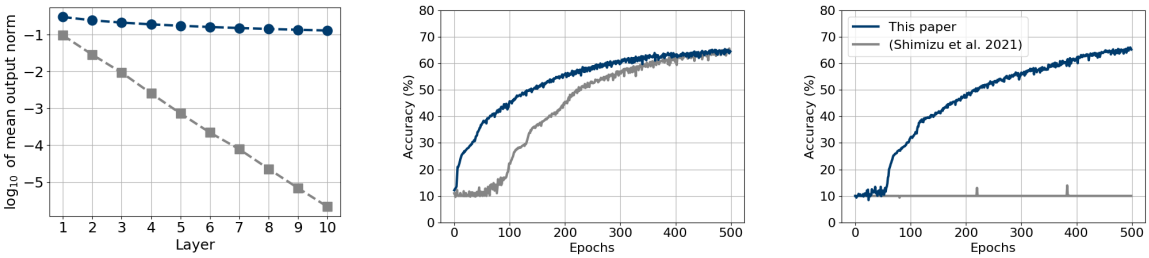
4. Experiments

We investigate (i) the effect of network initialization over many layers, (ii) the effect of curvature and ReLU activations, (iii) the comparison between Fréchet-based and our midpoint-based batch normalization and (iv) the robustness of hyperbolic residual networks. We seek to evaluate Poincaré ResNets in isolation and hence stick to minimal augmentation and fixed hyperparameters. We use random cropping and horizontal flipping with Adam optimization with fixed learning rate 10^{-3} and weight-decay 10^{-4}.

4.1. Identity initialization is norm-preserving

The approach of Shimizu *et al*. [47] is the current leading initialization in hyperbolic networks. This initialization,

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.



(a) Output norm over layers. (b) 5-layer ConvNet training. (c) 7-layer ConvNet training.

Figure 2: **Comparison to the initialization of [47].** In (a), we show the logarithm of the mean of the norms of each layer's output during the forward pass of an untrained 10-layer fully-connected network for random Poincaré gyrovectors. The figure shows that the baseline initialization is suffering from vanishing signals with outputs that collapse to the origin over multiple layers. Our identity-based initialization maintains output norms over layers. In (b) and (c), we show the test accuracy over epochs for a 5-layer and a 7-layer ConvNet. For a 5-layer network, the baseline initialization converges more slowly, while it no longer learns for 7-layers. Our initialization is preferred for training convolutional networks in the Poincaré ball model.

however, leads to vanishing signals, which we empirically validate here. We take a stack of 10 Poincaré linear layers with a constant curvature of $c = 1$, with 20 input and output neurons. We then perform a single forward pass on a batch of 16 Poincaré gyrovectors which are generated by sampling vectors in the tangent space at the origin from the multivariate normal distribution $\mathcal{N}(0, \frac{1}{10} I_{20})$ and mapping these to the Poincaré ball. Figure 2a shows the behaviour of the norms during the forward pass for both initialization methods. Where the baseline initialization suffers from vanishing signals, our identity initialization keeps the norms constant up to the rounding effects of the repeated application of non-linear operations.

In Figures 2b and 2c we additionally show what happens when training a simple ConvNet on CIFAR-10 with both initialization methods trained with SGD with learning rate 10^{-3}, momentum 0.9 and weight decay 10^{-4}. For a 5-layer ConvNet, the baseline initialization converges more slowly. For a 7-layer ConvNet, we find that the baseline is no longer capable of learning meaningful representations. Identity-based initialization is still able to train in this setting. We conclude that our identity-based hyperbolic network initialization is preferred for training hyperbolic networks.

4.2. Curvatures and ReLUs stabilize optimization

Previous works claim that nonlinearities, such as the ReLU operation, are redundant in hyperbolic neural networks due to the many nonlinearities inherent to such networks [16, 47]. Here, we test this claim by training a small Poincaré ResNet-20 on CIFAR-10 with small channel widths of (4, 8, 16) with and without the ReLU nonlinearity as activation layer. The results are shown in Figure 3 (left). We find that training with the ReLU nonlinearity leads to faster convergence and a greater final accuracy. This shows that nonlinear activation functions remain important despite the inherent nonlinearity of hyperbolic networks.

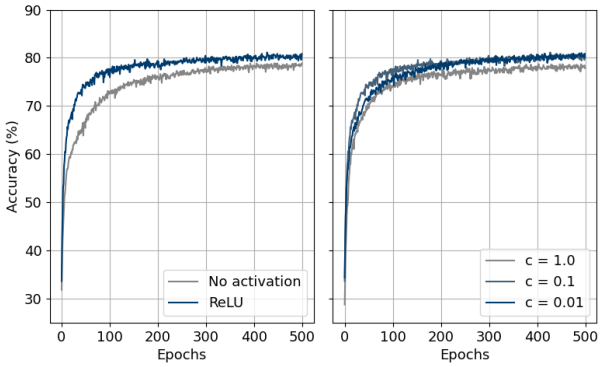


Figure 3: **ReLUs and small curvatures improve** the optimization and performance of Poincaré ResNet. Both experiments were performed using a small ResNet-20 with small channel widths (4, 8, 16). While hyperbolic layers are already non-linear, adding ReLUs further improves generalization. The same holds for using smaller curvatures.

Poincaré balls of various curvatures have similar geometric properties. For numerical computations however, setting the right curvature impacts the down-stream performance [17]. In this analysis, we investigate the effect of various curvatures for optimizing Poincaré ResNets. We again perform the experiments on a small Poincaré ResNet-20 with small channel widths of (4, 8, 16) using a curvature of 1, 0.1 or 0.01. We show the results in Figure 3 (right). We first find that training with a curvature of $c = 1$ leads to suboptimal accuracies. As the curvature becomes smaller, the Euclidean volume of the Poincaré ball increases. As a result, representing elements within this manifold using floating-point representations becomes easier with smaller

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.

	Manifold	CIFAR-10						CIFAR-100					
		FPR95 ↓		AUROC ↑		AUPR ↑		FPR95 ↓		AUROC ↑		AUPR ↑	
		R20	R32	R20	R32	R20	R32	R20	R32	R20	R32	R20	R32
Places-365	Euclidean	64.2	72.3	**84.7**	82.0	**96.2**	95.6	89.5	93.9	62.5	57.9	89.3	87.9
	w/ HNN++	**63.8**	72.7	79.6	77.7	94.5	94.2	93.2	86.3	63.3	66.6	89.8	91.1
	Poincaré	70.2	**70.7**	82.3	**82.6**	95.7	**95.9**	**82.8**	**83.8**	**71.5**	**71.1**	**92.3**	**92.2**
SVHN	Euclidean	97.3	94.7	68.8	73.4	92.8	94.1	99.5	98.8	43.7	54.6	83.7	88.2
	w/ HNN++	73.1	79.1	**85.5**	82.2	**96.9**	96.1	92.1	88.6	66.4	68.9	91.1	92.0
	Poincaré	**66.0**	**69.3**	85.0	**83.6**	96.6	**96.3**	**76.9**	**83.0**	**76.8**	**72.6**	**94.1**	**92.9**
Textures	Euclidean	87.3	88.0	73.6	77.3	93.2	94.7	98.1	96.0	33.5	42.9	75.9	79.4
	w/ HNN++	**63.8**	**56.6**	79.6	**85.8**	94.5	**96.6**	85.9	**77.5**	58.9	65.7	86.8	89.0
	Poincaré	68.2	66.2	**82.1**	82.3	**95.5**	95.6	**83.9**	84.2	**67.7**	**68.8**	**91.0**	**91.5**

Table 2: **Out-of-distribution detection** on CIFAR-10 and CIFAR-100 with Places365, SVHN, and DTD as out-of-distribution datasets. R20 and R32 denote ResNet-20 and ResNet-32 architectures, both with channel widths (8, 16, 32). Across different in- and out-of-distribution datasets, hyperbolic ResNets are more robust than their Euclidean counterparts.

		ResNet-20	ResNet-32
Accuracy	Fréchet mean	79.4	82.4
	Poincaré midpoint	80.9	81.9
Time (s epoch^{-1})	Fréchet mean	179.0	169.4
	Poincaré midpoint	137.5	132.0
		-23%	-22%

Table 1: **Poincaré midpoints for batch normalization** in hyperbolic space are as effective for classification as Fréchet means while being faster to optimize.

curvatures. Indeed, when training with curvatures $c = 0.1$ and $c = 0.01$, we find that the model converges faster and has a higher final accuracy. Overall, we find that a curvature of $c = 0.1$ works best for training Poincaré ResNets and we will use this setting for the rest of the experiments.

4.3. Midpoints make batch norm efficient

To compare the computational efficiency and the performance of our Poincaré midpoint batch normalization to the batch normalization by [34], we perform multiple experiments using Poincaré ResNet-20 or Poincaré ResNet-32 on CIFAR-10 with small channel widths of (4, 8, 16). We opt for a small ResNet width and fixed hyperparameters to allow for faster evaluation, all models obtain higher scores with more depth and hyperparameter tuning. Each network is then trained with Fréchet-based batch normalization [34] or with our Poincaré midpoint batch normalization.

The results of the experiment are shown in Table 1. First, we find that both batch normalization methods lead to similar accuracies, which indicates that Poincaré midpoints are as effective as Fréchet means for classification. Second, training a network with Poincaré midpoint batch normalization leads to a reduction in computation time of approximately 20-25%. We recommend Poincaré midpoints when performing batch normalization in hyperbolic networks.

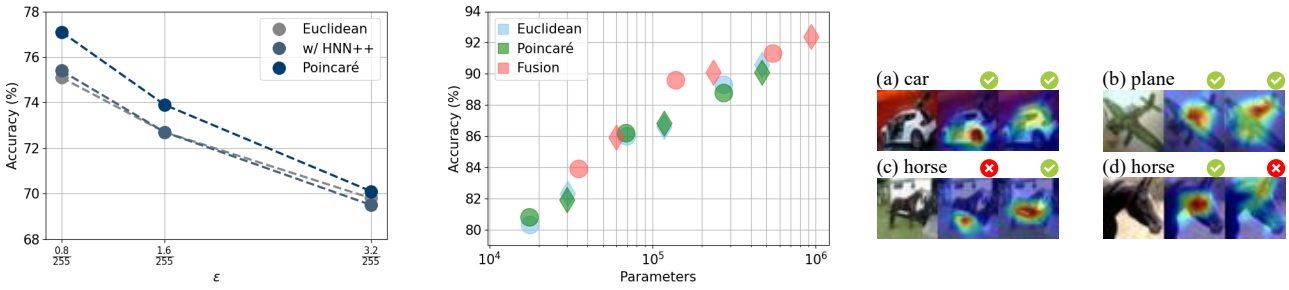
4.4. Hyperbolic networks are robust

Finally, we investigate the robustness and complementary nature of Poincaré ResNet compared to its Euclidean alternative. We investigate whether Poincaré ResNet is (i) robust to out-of-distribution samples, (ii) can handle adversarial examples, and (iii) learns complementary representations compared to Euclidean ResNet.

Out-of-distribution detection. To check whether Poincaré ResNets are robust to out-of-distribution samples, we compare the out-of-distribution detection performance of Euclidean and Poincaré ResNet-20 and ResNet-32 with channel widths (8, 16, 32), trained on either CIFAR-10 or CIFAR-100 using the same hyperparameters and optimizer as before and where the Euclidean ResNets have either a Euclidean or Poincaré classifier [47]. For each architecture, the Euclidean and hyperbolic variants have similar classification performance, hence any difference in out-of-distribution performance is not a result of improved training. We use the Places-365 dataset [61], the SVHN dataset [41] and the Textures dataset [8] as out-of-distribution datasets. For detecting out-of-distribution samples, we use the energy score as introduced by Liu *et al*. [31]. The comparisons are performed on the commonly used metrics FPR95, AUROC and AUPR.

The results are shown in Table 2. We find that with a ResNet-32 architecture, Poincaré ResNet outperforms both types of Euclidean ResNets on nearly all metrics for five out of six combinations of in- and out-of-distribution datasets. With a ResNet-20, Poincaré ResNet is better for each combination with CIFAR-100 as the in-distrution dataset. We conclude that a hyperbolic ResNet is more robust to out-of-distribution samples than its Euclidean counterpart, es-

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.



(a) Adversarial robustness. (b) Hyperbolic/Euclidean fusion. (c) Grad-CAM visualizations.

Figure 4: Comparisons and fusions between hyperbolic and Euclidean ResNets. (a) **Robustness to FGSM adversarial attacks** between Euclidean and Poincaré ResNets. These results are obtained by attacking a Poincaré ResNet-32, or a Euclidean ResNet-32 with either a Euclidean classifier or a Poincaré classifier [47], with small channel widths (8, 16, 32), trained on CIFAR-10 to similar performance, with adversarial examples of varying perturbation sizes ϵ. Poincaré ResNet is more robust to FGSM adversarial attacks. (b) **Fusion ResNets** plotted as a function of model parameters. The circle markers represent ResNet-20 and the diamond markers represent ResNet-32, with other differences due to varying channel widths of (4, 8, 16), (8, 16, 32) and (16, 32, 64). Fusing the Poincaré and Euclidean ResNets not only improves accuracy, but is more efficient than increasing the number of parameters of individual models, highlighting the strong complementary nature of learning visual representations in both spaces. (c) **Grad-CAM visualizations** of Euclidean (middle) and Poincaré (right) ResNets. (a) + (b) Both models predict the correct class while focusing on different discriminants in the image. (c) + (d) Failure case of respectively Euclidean and Poincaré ResNet due to a focus on ambiguous object parts.

pecially in the presence of many in-distribution classes.

Adversarial attacks. To see if Poincaré ResNet is robust to adversarial samples, we compare the performance against an adversarial attack between Euclidean ResNet-32 with either a Euclidean classifier or a Poincaré classifier [47] and Poincaré ResNet-32, each with channel widths (8, 16, 32), trained on CIFAR-10. Note that, after training, each model has similar performance on the test set of CIFAR-10. We apply the fast gradient signed method (FGSM) [24] attack with perturbations $\epsilon = \frac{0.8}{255}, \frac{1.6}{255}, \frac{2.4}{255}, \frac{3.2}{255}$ to the models. The results are shown in Figure 4a. We find that Poincaré ResNet is more resistant to adversarial attacks than the Euclidean ResNet, even though both architectures were trained similarly and obtained similar classification performance. This result highlights the potential of hyperbolic learning in the presence of adversarial agents. We note that Euclidean ResNets normally use running statistics, while Poincaré ResNets do not. Here we have disabled running statistics for the Euclidean models to ensure a fair comparison as running statistics make a model far more susceptible to adversarial attacks. The results when using running statistics are shown in the appendix.

Complementary representations. To show that the representations learned by Poincaré ResNets are complementary to the features from Euclidean ResNets, we evaluate the performance of a fusion model, where each image is forwarded through both ResNets and the resulting logits are averaged to obtain predictions. Note that both models are trained independently and the fusion model is only evaluated with no further training being performed. The results are shown in Figure 4b. For each architecture, the performance on both manifolds is similar. Clearly, the performance of the fusion models is better than that of the individual ResNets. With respect to the number of parameters, we find that it is more efficient to create a fusion model than it is to increase the size of whichever ResNet we are using. In Figure 4c, we also show Grad-CAM visualizations [46], highlighting that our approach focuses more on the different parts that form the object, instead of the single most discriminative component like in Euclidean ResNets.

5. Discussion

In this paper we propose Poincaré ResNet and make several contributions. First, we formulate the Poincaré residual block including convolutions, batchnorm, and ReLU's. Second, we introduce an initialization that prevents vanishing signals and allows for the training of deeper models. Third, we propose a new hyperbolic batch normalization based on the Poincaré midpoint, which substantially increases efficiency at no cost to its performance. Fourth, we manually derive the backward pass for several operations within the Poincaré ball to decrease the size of the computation graphs. Empirically, we perform initial explorations into fully hyperbolic neural networks, showing that Poincaré Resnets are (i) more robust to out-of-distribution samples, (ii) more robust to adversarial attacks and (iii) complementary to Euclidean networks.

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.

References

[1] Ivana Balazevic, Carl Allen, and Timothy Hospedales. Multi-relational poincaré graph embeddings. *Advances in Neural Information Processing Systems*, 2019. 1

[2] Silvere Bonnabel. Stochastic gradient descent on riemannian manifolds. *IEEE Transactions on Automatic Control*, 58:2217–2229, 2013. 5

[3] Joey Bose, Ariella Smofsky, Renjie Liao, Prakash Panangaden, and Will Hamilton. Latent variable modelling with hyperbolic normalizing flows. In *International Conference on Machine Learning*, 2020. 3

[4] Ines Chami, Albert Gu, Vaggos Chatziafratis, and Christopher Ré. From trees to continuous embeddings and back: Hyperbolic hierarchical clustering. In *Advances in Neural Information Processing Systems*, 2020. 1

[5] Ines Chami, Rex Ying, Christopher Ré, and Jure Leskovec. Hyperbolic graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, 2019. 1

[6] Bike Chen, Wei Peng, Xiaofeng Cao, and Juha Röning. Hyperbolic uncertainty aware semantic segmentation. *Transactions on Intelligent Transportation Systems*, 2022. 1, 3

[7] Weize Chen, Xu Han, Yankai Lin, Hexu Zhao, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. Fully hyperbolic neural networks. *arXiv preprint arXiv:2105.14686*, 2021. 1

[8] Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *Computer Vision and Pattern Recognition*, 2014. 7

[9] Jindou Dai, Yuwei Wu, Zhi Gao, and Yunde Jia. A hyperbolic-to-hyperbolic graph convolutional network. In *Computer Vision and Pattern Recognition*, 2021. 1

[10] Ankit Dhall, Anastasia Makarova, Octavian Ganea, Dario Pavllo, Michael Greeff, and Andreas Krause. Hierarchical image classification using entailment cone embeddings. In *Computer Vision and Pattern Recognition Workshops*, 2020. 1, 3

[11] Bhuwan Dhingra, Christopher J Shallue, Mohammad Norouzi, Andrew M Dai, and George E Dahl. Embedding text in hyperbolic spaces. In *Workshop on Graph-Based Methods for Natural Language Processing*, 2018. 1

[12] Aleksandr Ermolov, Leyla Mirvakhabova, Valentin Khrulkov, Nicu Sebe, and Ivan Oseledets. Hyperbolic vision transformers: Combining improvements in metric learning. In *Computer Vision and Pattern Recognition*, 2022. 1, 3

[13] Pengfei Fang, Mehrtash Harandi, and Lars Petersson. Kernel methods in hyperbolic spaces. In *International Conference on Computer Vision*, 2021. 1, 3

[14] Luca Franco, Paolo Mandica, Bharti Munjal, and Fabio Galasso. Hyperbolic self-paced learning for self-supervised skeleton-based action representations. In *International Conference on Learning Representations*, 2023. 1

[15] Octavian Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic entailment cones for learning hierarchical embeddings. In *International Conference on Machine Learning*, 2018. 1

[16] Octavian-Eugen Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic neural networks. In *Advances in Neural Information Processing Systems*, 2018. 1, 2, 6

[17] Zhi Gao, Yuwei Wu, Yunde Jia, and Mehrtash Harandi. Curvature generation in curved spaces for few-shot learning. In *International Conference on Computer Vision*, 2021. 1, 3, 6

[18] Mina Ghadimi Atigh, Martin Keller-Ressel, and Pascal Mettes. Hyperbolic busemann learning with ideal prototypes. In *Advances in Neural Information Processing Systems*, 2021. 1, 3

[19] Mina Ghadimi Atigh, Julian Schoep, Erman Acar, Nanne van Noord, and Pascal Mettes. Hyperbolic image segmentation. In *Computer Vision and Pattern Recognition*, 2022. 1, 3

[20] Yunhui Guo, Xudong Wang, Yubei Chen, and Stella X Yu. Clipped hyperbolic classifiers are super-hyperbolic classifiers. In *Computer Vision and Pattern Recognition*, 2022. 1, 3

[21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *International Conference on Computer Vision*, 2015. 5

[22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Computer Vision and Pattern Recognition*, 2016. 1, 3, 4

[23] Joy Hsu, Jeffrey Gu, Gong Wu, Wah Chiu, and Serena Yeung. Capturing implicit hierarchical structure in 3d biomedical images with self-supervised hyperbolic representations. *Advances in Neural Information Processing Systems*, 2021. 3

[24] Christian Szegedy Ian Goodfellow, Jonathan Shlens. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015. 8

[25] Valentin Khrulkov, Leyla Mirvakhabova, Evgeniya Ustinova, Ivan Oseledets, and Victor Lempitsky. Hyperbolic image embeddings. In *Computer Vision and Pattern Recognition*, 2020. 1, 3

[26] Anna Klimovskaia, David Lopez-Paz, Léon Bottou, and Maximilian Nickel. Poincaré maps for analyzing complex hierarchies in single-cell data. *Nature communications*, 11, 2020. 1

[27] Marc Law, Renjie Liao, Jake Snell, and Richard Zemel. Lorentzian distance learning for hyperbolic representations. In *International Conference on Machine Learning*, 2019. 1

[28] Diego Lazcano, Nicolás Fredes Franco, and Werner Creixell. Hgan: Hyperbolic generative adversarial network. *IEEE Access*, 9:96309–96320, 2021. 3

[29] Qi Liu, Maximilian Nickel, and Douwe Kiela. Hyperbolic graph neural networks. In *Advances in neural information processing systems*, 2019. 1

[30] Shaoteng Liu, Jingjing Chen, Liangming Pan, Chong-Wah Ngo, Tat-Seng Chua, and Yu-Gang Jiang. Hyperbolic visual embedding learning for zero-shot recognition. In *Computer Vision and Pattern Recognition*, 2020. 3

[31] Weitang Liu, Xiaoyun Wang, John Owens, and Yixuan Li. Energy-based out-of-distribution detection. In *Advances in neural information processing systems*, 2020. 7

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.

[32] Teng Long, Pascal Mettes, Heng Tao Shen, and Cees G M Snoek. Searching for actions on the hyperbole. In *Computer Vision and Pattern Recognition*, 2020. 1, 3

[33] Teng Long and Nanne van Noord. Cross-modal scalable hyperbolic hierarchical clustering. In *ICCV*, 2023. 1

[34] Aaron Lou, Isay Katsman, Qingxuan Jiang, Serge J. Belongie, Ser-Nam Lim, and Christopher De Sa. Differentiating through the fréchet mean. In *International Conference on Machine Learning*, 2020. 1, 4, 5, 7

[35] Rongkai Ma, Pengfei Fang, Tom Drummond, and Mehrtash Harandi. Adaptive poincaré point to set distance for few-shot classification. In *AAAI Conference on Artificial Intelligence*, 2022. 1, 3

[36] Emile Mathieu, Charline Le Lan, Chris J Maddison, Ryota Tomioka, and Yee Whye Teh. Continuous hierarchical representations with poincaré variational auto-encoders. In *Advances in neural information processing systems*, 2019. 3

[37] Emile Mathieu and Maximilian Nickel. Riemannian continuous normalizing flows. *Advances in Neural Information Processing Systems*, 2020. 3

[38] Pascal Mettes, Mina Ghadimi Atigh, Martin Keller-Ressel, Jeffrey Gu, and Serena Yeung. Hyperbolic deep learning in computer vision: A survey. *arXiv preprint arXiv:2305.06611*, 2023. 3

[39] Nicholas Monath, Manzil Zaheer, Daniel Silva, Andrew McCallum, and Amr Ahmed. Gradient-based hierarchical clustering using continuous representations of trees in hyperbolic space. In *International Conference on Knowledge Discovery & Data Mining*, 2019. 3

[40] Yoshihiro Nagano, Shoichiro Yamaguchi, Yasuhiro Fujita, and Masanori Koyama. A wrapped normal distribution on hyperbolic space for gradient-based learning. In *International Conference on Machine Learning*, 2019. 3

[41] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. Reading digits in natural images with unsupervised feature learning. In *NeurIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011. 7

[42] Maximillian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. *Advances in neural information processing systems*, 2017. 1

[43] Wei Peng, Tuomas Varanka, Abdelrahman Mostafa, Henglin Shi, and Guoying Zhao. Hyperbolic deep neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44:10023 – 10044, 2022. 2

[44] Frederic Sala, Chris De Sa, Albert Gu, and Christopher Ré. Representation tradeoffs for hyperbolic embeddings. In *International conference on machine learning*, 2018. 1

[45] Rik Sarkar. Low distortion delaunay embedding of trees in hyperbolic plane. In *International Conference on Graph Drawing*, 2011. 1

[46] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, 2017. 8

[47] Ryohei Shimizu, Yusuke Mukuta, and Tatsuya Harada. Hyperbolic neural networks++. In *International Conference on Learning Representations*, 2021. 1, 2, 3, 4, 5, 6, 7, 8

[48] Dídac Surís, Ruoshi Liu, and Carl Vondrick. Learning the predictability of the future. In *Computer Vision and Pattern Recognition*, 2021. 3

[49] Alexandru Tifrea, Gary Bécigneul, and Octavian-Eugen Ganea. Poincar\'e glove: Hyperbolic word embeddings. In *International Conference on Learning Representations*, 2019. 1

[50] Abraham Albert Ungar. *A Gyrovector Space Approach to Hyperbolic Geometry*. Springer Cham, 2009. 2, 4

[51] Abhinav Valada. On hyperbolic embeddings in object detection. In *German Conference on Pattern Recognition*, 2022. 3

[52] Max van Spengler, Philipp Wirth, and Pascal Mettes. HypLL: The hyperbolic learning library. In *ACM MM 2023 Open Source Competition*, 2023. 2

[53] Zhenzhen Weng, Mehmet Giray Ogut, Shai Limonchik, and Serena Yeung. Unsupervised discovery of the long-tail in instance segmentation using hierarchical self-supervision. In *Computer Vision and Pattern Recognition*, 2021. 3

[54] Yan Xu, Lifu Mu, Zhong Ji, Xiyao Liu, and Jungong Han. Meta hyperbolic networks for zero-shot learning. *Neurocomputing*, 491:57–66, 2022. 3

[55] Jiexi Yan, Lei Luo, Cheng Deng, and Heng Huang. Unsupervised hyperbolic metric learning. In *Computer Vision and Pattern Recognition*, 2021. 3

[56] Zhen Yu, Toan Nguyen, Yaniv Gal, Lie Ju, Shekhar S Chandra, Lei Zhang, Paul Bonnington, Victoria Mar, Zhiyong Wang, and Zongyuan Ge. Skin lesion recognition with class-hierarchy regularized hyperbolic embeddings. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, 2022. 1, 3

[57] Baoquan Zhang, Hao Jiang, Shanshan Feng, Xutao Li, Yunming Ye, and Rui Ye. Hyperbolic knowledge transfer with class hierarchy for few-shot learning. In *International Joint Conference on Artificial Intelligence*, 2022. 1, 3

[58] Huanqiu Zhang, P Dylan Rich, Albert K Lee, and Tatyana O Sharpee. Hippocampal spatial representations exhibit a hyperbolic geometry that expands with experience. *Nature Neuroscience*, 26:131–139, 2022. 1

[59] Yanfu Zhang, Lei Luo, Wenhan Xian, and Heng Huang. Learning better visual data similarities via new grouplet noneuclidean embedding. In *International Conference on Computer Vision*, 2021. 1, 3

[60] Yiding Zhang, Xiao Wang, Chuan Shi, Xunqiang Jiang, and Yanfang Ye. Hyperbolic graph attention network. *IEEE Transactions on Big Data*, 8:1690–1701, 2021. 1

[61] Bolei Zhou, Agata Lapedriza, Aditya Khosla, Aude Oliva, and Antonio Torralba. Places: A 10 million image database for scene recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40:1452–1464, 2017. 7

Authorized licensed use limited to: National Cheng Chi University. Downloaded on October 18,2026 at 16:29:02 UTC from IEEE Xplore. Restrictions apply.